

TRABAJO PRÁCTICO INTEGRADOR

Cátedra de Ingeniería de Software — Modalidad Taller en Clase

Fecha: Julio 2026

Modalidad: Grupal (2-3 integrantes)

Duración: 90 - 120 minutos

ASIGNATURA	TEMA PRINCIPAL	INTEGRANTES DEL GRUPO
Ingeniería de Software	Requerimientos, Ciclo de Vida y Pruebas	1. _____ 2. _____ 3. _____

Caso de Estudio: Sistema de Reservas en Línea para Cine ("CineMax")

Una importante cadena de cines desea desarrollar una plataforma web y móvil integral para que sus clientes puedan consultar la cartelera actualizada, comprar entradas de forma anticipada y seleccionar sus asientos en tiempo real mediante un mapa interactivo de la sala. Asimismo, el gerente de la sucursal requiere acceder a un módulo de administración donde pueda visualizar métricas de ventas, ocupación de salas diariamente y gestionar la programación de películas.

PARTE 1 Especificación y Análisis de Requerimientos (SRS)

- Requerimientos del Usuario vs. Requerimientos del Sistema:**
 - Redacten **2 Requerimientos del Usuario** formulados en lenguaje natural y orientados a la necesidad del cliente o del gerente.
 - A partir de cada requerimiento de usuario anterior, especifiquen **2 Requerimientos del Sistema** técnicos y detallados que sirvan de guía directa para el equipo de desarrollo.
- Clasificación Requerimientos Funcionales (RF) y No Funcionales (RNF):**
 - Formulen una lista de **4 Requerimientos Funcionales (RF)** clave para el sistema.
 - Formulen **3 Requerimientos No Funcionales (RNF)**. *Obligatorio:* Al menos uno debe corresponder al atributo de **Rendimiento/Eficiencia** y otro a **Seguridad**.
- Modelado con Casos de Uso (UML):**
 - Diseñen un diagrama de Casos de Uso que incluya al menos dos actores principales (ej: Cliente y Administrador/Gerente) y un mínimo de 4 casos de uso clave (ej: *Reservar Asiento*, *Procesar Pago*, *Generar Reporte de Ventas*). Indiquen la relación entre los actores y los casos de uso.

PARTE 2 Ciclo de Vida, Arquitectura y Programación

- Paradigma Orientado a Objetos (POO):**
 - Justifiquen brevemente por qué la Programación Orientada a Objetos es adecuada para modelar este dominio de negocio.
 - Definan **2 Clases principales** del dominio (ej: Entrada, Funcion, Asiento, Cliente). Para cada clase, detallen:
 - Al menos **2 atributos** con su tipo de dato correspondiente.
 - Al menos **1 método principal** describiendo su responsabilidad.

2. Estrategia de Control de Versiones (Git):

- Supongan que el equipo de desarrollo está conformado por 4 programadores trabajando simultáneamente en distintas partes del sistema (Backend, Frontend Móvil, Pasarela de Pagos y Módulo de Reportes). Expliquen qué estrategia de ramas (branching) y buenas prácticas en Git utilizarían para evitar sobreescribir código y asegurar integraciones fluidas.

PARTE 3 Planificación de Pruebas y Calidad de Software (Testing)

1. Matriz de Pruebas por Nivel: Completen el siguiente cuadro definiendo un escenario de prueba concreto y adaptado al sistema "CineMax" para cada uno de los niveles indicados:

Nivel de Prueba	Escenario / Ejemplo Concreto en "CineMax"	Criterio de Aceptación / Resultado Esperado
Prueba Unitaria	(Ej: Verificar la lógica de cálculo del precio final aplicando descuentos por día...)	(Comprobar que el método retornarMontoTotal() devuelve el valor correcto)
Prueba de Integración	(Ej: Validar la comunicación entre el módulo de reserva y la API externa de pago...)	...
Prueba de Sistema	(Ej: Ejecutar el flujo completo de selección, compra y emisión de ticket digital...)	...
Prueba de Aceptación (UAT)	(Ej: El gerente de la sucursal prueba la generación de reportes diarios...)	...

1. Diseño de Tipos de Pruebas Específicas:

- **Prueba de Carga y Estrés:** Diseñen un escenario de prueba de rendimiento simulando el día de estreno de una película taquillera. Especifiquen qué métricas o variables clave van a medir (ej: tiempo de respuesta, tasa de errores HTTP, latencia de base de datos).
- **Caja Negra vs. Caja Blanca:** Expliquen la diferencia práctica entre aplicar una prueba de *Caja Negra* y una de *Caja Blanca* sobre el algoritmo que valida la selección de asientos contiguos disponibles.

📋 Criterios de Evaluación y Dinámica de Puesta en Común

- **Claridad y Precisión (30%):** Correcta redacción y estructuración de los requerimientos sin ambigüedades.
- **Coherencia de Diseño (35%):** Alineación entre el caso de uso, el modelo de clases en POO y las reglas de negocio planteadas.
- **Estrategia de QA y Testing (35%):** Correcta clasificación de los niveles de prueba y rigor técnico al definir el escenario de estrés.
- **Puesta en Común (Últimos 20 min):** Cada equipo expondrá brevemente un RNF de rendimiento definido y la estrategia de prueba de estrés diseñada.